

系统开发工具基础 · 实验报告

第 1 周：课程概览与 Shell 入门 · Version Control and Git · LaTeX 文档编辑

姓 名	徐宜安	学 号	25060012033
授课教师	周小伟	上课日期	2026 年 8 月 24 日
Git 仓库	GitHub 课程仓库	报告日期	2026 年 8 月 31 日

1 课上练习

1.1 q01 · 含空格文件名的批量整理 (Shell 基础与文件系统)

题目要求

在当前目录新建 q01 并完全使用命令行完成：

- 创建 input/docs 与 input/tmp，并创建四个文件：notes one.txt (alpha、beta 两行)、.secret.txt (隐藏文件)、empty.txt (空文件)、run.log (任意两行日志)；
- 显示 q01 的绝对路径，以长格式列出 input 下全部项目，包含隐藏文件；
- 把 input 下所有 .txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理名称中的空格；
- 复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

实现思路：含空格的文件名必须作为一个完整参数传递；如果不加引号，Shell 会把 notes one.txt 拆成两个参数。脚本使用 find -exec cp --parents 逐个复制匹配到的文件，并用 --parents 保留原目录结构。随后分别对目录和普通文件设置 750、640 权限，再用 -printf "%P %s" 生成相对路径与字节数清单。完整脚本见代码 1，分步执行过程见图 1。

```
1  #!/bin/bash
2  set -euo pipefail
3
4  STUDENT_ID="25060012033"
5
6  mkdir -p q01
7  cd q01
8
9  # 创建文件夹
10 mkdir -p input/docs input/tmp
11
```

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础 (main)
● $ cd ./第一周/课上

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上 (main)
● $ mkdir q01

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上 (main)
● $ cd q01/

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ mkdir -p input/docs input/tmp

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ echo -e "alpha\nbeta" > "input/docs/notes one.txt"

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ echo "hidden" > input/docs/.secret.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ touch input/tmp/empty.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ echo -e "line one log\nline two log" > input/run.log

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ pwd
/d/desktop/系统开发工具基础/第一周/课上/q01

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ ls -lAR input
input:
total 1
drwxr-xr-x 1 xya 197121 0 8月 30 21:02 docs/
-rw-r--r-- 1 xya 197121 28 8月 30 21:02 run.log
drwxr-xr-x 1 xya 197121 0 8月 30 21:02 tmp/

input/docs:
total 2
-rw-r--r-- 1 xya 197121 8 8月 30 21:02 .secret.txt
-rw-r--r-- 1 xya 197121 13 8月 30 21:02 'notes one.txt'

input/tmp:
total 0
-rw-r--r-- 1 xya 197121 0 8月 30 21:02 empty.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ mkdir -p work/25060012033

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ find input -type f -name "*.txt" -exec cp --parents {} work/25060012033/ \;

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01 (main)
● $ cd work/25060012033/

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01/work/25060012033 (main)
● $ find . -type d -exec chmod 750 {} \;

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01/work/25060012033 (main)
● $ find . -type f -exec chmod 640 {} \;

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q01/work/25060012033 (main)
● $ find . -type f -printf "%P %s\n" > ../../inventory.txt
```

图 1 q01 分步命令执行过程（终端截图）

```
12 # 创建各个文件
13 echo -e "alpha\nbeta" > "input/docs/notes one.txt"
14 echo "hidden" > input/docs/.secret.txt
15 touch input/tmp/empty.txt
16 echo -e "log line 1\nlog line 2" > input/run.log
17
18 # 打印绝对路径，递归列出 input 下全部内容（包含隐藏文件）
19 pwd
20 ls -lAR input
21
22 # 复制文件，保留目录结构，find方案（兼容空格文件名）
23 mkdir -p work/${STUDENT_ID}
24 find input -type f -name "*.txt" -exec cp --parents {} work/${STUDENT_ID}/ \;
25
26 # 修改权限
27 cd work/${STUDENT_ID}
28 find . -type d -exec chmod 750 {} \;
29 find . -type f -exec chmod 640 {} \;
30
31 # 生成inventory.txt，相对路径+字节数
32 find . -type f -printf "%P %s\n" > ../../inventory.txt
33 cd ../../
34
35 echo "任务完成！"
```

代码 1 q01.sh：批量整理总脚本

结果：复制后的目录结构见代码 2。run.log 因扩展名不符而未复制，三个 .txt 文件均保留了原目录结构。inventory.txt 的结果见代码 3，三个文件的字节数分别为 7、11 和 0；目录权限设为 750，文件权限设为 640。图中重新执行 ls 时显示为 8、13 和 0 字节，是因为文件后来使用了 Windows 的 CRLF 换行；清单生成时使用的是 LF 换行，可见文本内容没有变化。

```
1 q01/
2 |-- input/
3 |   |-- docs/
4 |   |   |-- .secret.txt      # 隐藏文件
5 |   |   |-- notes one.txt   # 含空格文件名
6 |   |-- tmp/
7 |   |   |-- empty.txt       # 空文件
8 |   |-- run.log
9 |-- work/
```

```
10 | `-- 25060012033/
11 |     `-- input/                # 仅 .txt 被复制
12 |         |-- docs/            # .secret.txt, notes one.txt
13 |         `-- tmp/             # empty.txt
14 |-- inventory.txt
15 `-- q01.sh
```

代码 2 q01 最终目录结构 (work/ 下仅复制.txt 文件)

```
1 input/docs/.secret.txt 7
2 input/docs/notes one.txt 11
3 input/tmp/empty.txt 0
```

代码 3 inventory.txt: 复制清单 (相对路径 + 字节数)

1.2 q02 · 随机访问日志统计 (Shell 管道与文本处理)

题目要求

在 q02 中创建 access.csv，并完成日志统计：

- 编写 analyze.sh，接收一个 CSV 文件路径；文件不存在时将错误写入 stderr 并返回非零退出码；
- 使用 Shell 管道以及 awk、sort、head 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序；
- 输出全部数据行的平均 latency_ms，保留两位小数；表头不得计入；
- 分别对 access.csv 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

实现思路：脚本先检查输入文件是否存在，不存在时向 stderr 输出错误并以状态码 1 退出。统计部分分为三步：

- 过滤与提取：awk -F',' 'NR>1 && \$4 ~ /^5/ {print \$3}'——逗号分隔、跳过表头、status 以 5 开头时输出 path；
- 计数与排序：sort | uniq -c | sort -k1,1nr -k2,2 | head -n 2——先按次数数值降序、次数相同时按 path 字典序，取前 2；
- 平均值：awk 累加 \$5，END 块用 printf "%.2f" 保留两位小数，表头由 NR>1 排除。

完整实现见代码 4。

```
1 #!/bin/bash
2 set -euo pipefail
3
4 #接收第一个参数：csv文件路径
```

```
5 CSV="$1"
6
7 # 判断文件是否存在；不存在输出错误到stderr，退出码非0
8 if [[ ! -f "${CSV}" ]]; then
9     echo "error: 文件 ${CSV} 不存在" >&2
10    exit 1
11 fi
12
13 echo "==== 5xx最多的前2个path（次数降序，次数相同按path字典序）===="
14
15 # -F, 逗号分隔；NR>1跳过表头；status($4)以5开头；统计path；排序取前2
16 awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "${CSV}" \
17 | sort \
18 | uniq -c \
19 | sort -k1,1nr -k2,2 \
20 | head -n 2
21
22 echo -e "\n==== 全部数据行平均latency_ms（保留两位小数）===="
23
24 # NR>1跳过表头，累加latency总和、计数；用awk做浮点计算保留2位
25 awk -F',' 'NR>1 {sum += $5; cnt+=1} END {printf "%.2f\n", sum/cnt}' "${CSV}"
```

代码 4 analyze.sh: 访问日志统计脚本

结果:运行结果见图 2。5xx 数量最多的两个 path 是 /api/orders(3 次)和 /api/users(2 次)，8 条记录的平均延迟为 193.75 ms。传入不存在的文件时，脚本向 stderr 报错并返回状态码 1。



```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q02 (main)
$ chmod +x analyze.sh

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q02 (main)
$ ./analyze.sh access.csv
==== 5xx最多的前2个path（次数降序，次数相同按path字典序）====
  3 /api/orders
  2 /api/users

==== 全部数据行平均latency_ms（保留两位小数）====
193.75

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q02 (main)
$ ./analyze.sh nofile.csv
error: 文件 nofile.csv 不存在

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课上/q02 (main)
$ echo $?
1
```

图 2 q02 运行结果：5xx 统计、平均延迟与错误分支（终端截图）

1.3 q03 · 制造、解决并解释一次合并冲突 (Version Control and Git)

题目要求

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突：

- 初始化仓库，在 main 创建 config.txt (mode=normal) 并完成初始提交；
- 创建 feature-a 改为 mode=safe 并提交；从初始 main 创建 feature-b 改为 mode=fast 并提交；
- 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed；
- 确认工作区干净，运行 `git log --all --graph --decorate --oneline` 查看历史。

实验步骤如下（完整终端记录见图 3）：

1. **初始提交**：`git init` 建库，创建 config.txt (mode=normal) 并提交 (30c2b45)；
2. **两条功能分支**：从 main 分别创建 feature-a 和 feature-b，将 mode 改为 safe、fast 后提交；
3. **两次合并**：回 main 先合并 feature-a——此时 main 没有新提交，Git 走 Fast-forward 快进合并，无冲突；再合并 feature-b——两条分支修改了同一文件的同一行，触发内容冲突；
4. **解决并验证**：保留 mode=safe，追加 note=reviewed 后提交；再用 `git status` 和图形化日志检查结果。

```
1  git init
2  git add config.txt && git commit -m "init: mode=normal"
3
4  git checkout -b feature-a          # 从 main 分叉
5  # config.txt 改为 mode=safe
6  git commit -am "feature-a: set mode=safe"
7
8  git checkout main && git checkout -b feature-b  # 回到 main 再分叉
9  # config.txt 改为 mode=fast
10 git commit -am "feature-b: set mode=fast"
11
12 git checkout main
13 git merge feature-a                # Fast-forward, 无冲突
14 git merge feature-b                # 同一文件同一行 -> 内容冲突
```

代码 5 q03 关键命令：分支、合并与冲突解决

两条分支修改了 config.txt 的同一行，Git 因此写入冲突标记（代码 6）。我删除标记，保留 mode=safe，补上 note=reviewed，再暂存并提交（代码 7）。

```
1 <<<<<< HEAD
2 mode=safe
3 =====
4 mode=fast
5 >>>>>> feature-b
```

代码 6 冲突标记（解决前）

```
1 git add config.txt
2 git commit -m "merge feature-b: resolve conflict, keep safe, add note"
3 git status                                # working tree clean
4 git log --all --graph --decorate --oneline
```

代码 7 q03 冲突解决与验证

最终提交历史见代码 8: main 先快进到 feature-a, 随后通过合并提交整合 feature-b; git status 显示工作区干净。

```
1 * 34485a4 (HEAD -> main) merge feature-b: resolve conflict, keep safe, add
   note
2 |\
3 | * 9ed7505 (feature-b) feature-b: set mode=fast
4 * | 0affc5f (feature-a) feature-a: set mode=safe
5 |/
6 * 30c2b45 init: mode=normal
```

代码 8 合并后的图形化提交历史

1.4 q04 · 修复并构建一页技术说明 (LaTeX 文档编辑)

题目要求

将给定模板保存为 q04/report.tex, 补全内容并从命令行构建 PDF:

- 加入一个带编号和 label 的公式, 并在正文中使用 ref 或 eqref 引用;
- 加入一个 2 行 2 列表格, 设置 caption 和 label, 并在正文中引用该表;
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf;
- 确认 PDF 能打开, 且交叉引用不显示 “??”。

实现内容: 在 equation 环境中加入质能方程, 并用 \eqref 引用公式编号; 随后加入 2 行 2 列表格, 并在正文中引用表格。补全后的源文件见代码 9, 构建命令见代码 10。

```
1 \documentclass{article}
2 \usepackage[UTF8]{ctex}
```



```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ echo "mode=normal" > config.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git commit -m "init: mode=normal"
[main (root-commit) 30c2b45] init: mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git checkout -b feature-a
Switched to a new branch 'feature-a'

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-a)
● $ echo "mode=safe" > config.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-a)
● $ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-a)
● $ git commit -m "feature-a: set mode=safe"
[feature-a 0affc5f] feature-a: set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-a)
● $ git checkout main
Switched to branch 'main'

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git checkout -b feature-b
Switched to a new branch 'feature-b'

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-b)
● $ echo "mode=fast" > config.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-b)
● $ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-b)
● $ git commit -m "feature-b: set mode=fast"
[feature-b 9ed7505] feature-b: set mode=fast
1 file changed, 1 insertion(+), 1 deletion(-)

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (feature-b)
● $ git checkout main
Switched to branch 'main'

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git merge feature-a
Updating 30c2b45..0affc5f
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main|MERGING)
● $ git add config.txt

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main|MERGING)
● $ git commit -m "merge feature-b, resolve conflict: keep safe, add note"
[main 34485a4] merge feature-b, resolve conflict: keep safe, add note

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/q03 (main)
● $ git log --all --graph --decorate --oneline
* 34485a4 (HEAD -> main) merge feature-b, resolve conflict: keep safe, add note
| \
| * 9ed7505 (feature-b) feature-b: set mode=fast
| * 0affc5f (feature-a) feature-a: set mode=safe
| /
* 30c2b45 init: mode=normal
```

图 3 q03 分支创建、合并冲突与解决的完整过程（终端截图）


```
3 \usepackage{amsmath}
4 \begin{document}
5 \title{Tool Report}
6 \author{徐宜安-25060012033}
7 \date{2026 年 8 月 24 日}
8 \maketitle
9 \section{Result}
10 % 在此加入公式、表格及正文引用
11 下面展示核心计算公式，见公式~\ref{eq:energy}。
12 \begin{equation}
13 E = mc^2
14 \label{eq:energy}
15 \end{equation}
16 公式\eqref{eq:energy}是质能方程。
17
18 实验数据见表~\ref{tab:data}。
19 \begin{table}[h]
20 \centering
21 \caption{实验样本数据}
22 \label{tab:data}
23 \begin{tabular}{cc}
24 a & b \\
25 10 & 20
26 \end{tabular}
27 \end{table}
28 \end{document}
```

代码 9 report.tex: 补全后的技术说明

```
1 latexmk -pdf -halt-on-error report.tex
```

代码 10 q04 构建命令 (TeX Live 2025)

编译检查:

- label 分别放在公式环境内和表格 caption 之后;
- 使用 latexmk 自动完成多轮编译，避免交叉引用显示为 “??”。

结果: report.pdf 可正常打开，公式和表格的编号、正文引用均正常，未出现“??”(图 4)。

2 课后练习

参考资料记录 [Missing Semester: Shell 入门](#)、[Missing Semester: Shell 工具和脚本](#)、[Missing](#)

Tool Report

徐宜安-25060012033

2026 年 8 月 24 日

1 Result

下面展示核心计算公式，见公式 1。

$$E = mc^2 \quad (1)$$

公式(1)是质能方程。

实验数据见表 1。

表 1: 实验样本数据

a	b
10	20

图 4 q04 构建产物 report.pdf 的主要内容

Semester: [Version Control and Git](#)、OI Wiki: [LaTeX 入门](#)。

2.1 Shell 练习

1. `pwd`: 查看当前工作目录，输出绝对路径。
2. `ls -lah`: 以易读格式列出普通文件、隐藏文件和文件大小。
3. `mkdir -p lab/{src,logs,backup}`: 一次创建多层练习目录。
4. `printf "alpha\nbeta \n " > data.txt`: 生成两行文本，并用重定向写入文件。
5. `cp --parents src/data.txt backup/`: 复制文件并保留相对目录结构。
6. `mv data.txt src/input.txt`: 移动文件并同时完成重命名。
7. `find lab -type f -name "*.txt"`: 递归查找所有 txt 文件。
8. `chmod 640 src/input.txt`: 设置所有者可读写、同组只读的权限。
9. `wc -l -w -c src/input.txt`: 同时统计行数、单词数和字节数。
10. `grep -nE "alpha|beta" src/input.txt`: 使用扩展正则查找并显示行号。
11. `sort names.txt | uniq -c | sort -nr`: 统计重复项并按次数降序排列。
12. `cut -d',' -f1,3 scores.csv`: 按逗号分隔提取 CSV 的第 1、3 列。

13. `awk -F',' 'NR>1 {sum+=$2} END {print sum}' scores.csv`: 跳过表头并计算第二列总和。
14. `sed -E 's/[0-9]+/NUM/g' mixed.txt`: 用正则批量替换文本中的数字。
15. `tail -n 20 app.log | head -n 5`: 从日志末尾 20 行中取前 5 行。
16. `for f in *.txt; do echo "$f"; done`: 用循环逐个处理匹配到的文件。
17. `latest=$(ls -t *.log | head -n 1)`: 使用命令替换取得最新日志文件名。
18. `bash script.sh >stdout.log 2>stderr.log`: 分别保存标准输出和标准错误。
19. `echo $?`: 检查上一条命令的退出状态, 0 表示执行成功。
20. `du -ah lab | sort -h`: 统计目录占用空间并按大小排序。

练习与结果: 通过 Shell 中 `grep`、`sed`、`tr`、`sort`、`uniq`、`awk` 等命令完成检索、状态替换和词频统计, 最终得到 8 行、24 个单词, 其中 Git、Shell 和 LaTeX 分别出现 4、3、2 次, 相关命令及输出见图 5。

2.2 Git 练习

1. `git init git-practice`: 创建独立练习仓库。
2. `git status --short`: 用简洁格式查看工作区与暂存区状态。
3. `git add README.md`: 把指定文件加入暂存区。
4. `git diff`: 查看尚未暂存的修改。
5. `git diff --staged`: 查看即将进入下一次提交的修改。
6. `git add -p`: 按修改块交互式暂存, 拆分不同目的的改动。
7. `git commit -m "Add initial notes"`: 创建一次内容集中的提交。
8. `git log --all --graph --decorate --oneline`: 以图形方式查看提交 DAG 和分支指针。
9. `git switch -c experiment`: 创建并切换到实验分支。
10. `git merge --no-ff experiment`: 保留一次明确的合并提交。
11. `git stash push -m "unfinished notes"`: 临时保存未完成的修改。
12. `git stash list`: 查看暂存的工作现场。
13. `git stash pop`: 恢复最近一次 stash 并从列表中移除。
14. `git restore README.md`: 丢弃指定文件尚未暂存的修改。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础 (main)
• $ cd 第一周/

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周 (main)
• $ mkdir -p "课后/shell-notes"/{daily,result}

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周 (main)
• $ cd "课后/shell-notes"

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ printf "TODO: learn Shell\nGit branch practice\nShell pipeline\n" > daily/monday.txt; printf "TODO: review Git\nLaTeX document practice\nGit
stash\n" > daily/tuesday.txt; printf "TODO: write LaTeX report\nShell and Git review\n" > daily/wednesday.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ pwd
/d/desktop/系统开发工具基础/第一周/课后/shell-notes

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ ls -lah daily
total 3.0K
drwxr-xr-x 1 xya 197121  0  8月 30 23:20 ./
drwxr-xr-x 1 xya 197121  0  8月 30 23:17 ../
-rw-r--r-- 1 xya 197121 53  8月 30 23:21 monday.txt
-rw-r--r-- 1 xya 197121 51  8月 30 23:21 tuesday.txt
-rw-r--r-- 1 xya 197121 46  8月 30 23:21 wednesday.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ cat daily/*.txt > result/all-notes.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ wc -l -w -c result/all-notes.txt
  8  24 150 result/all-notes.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ grep -nEi "shell|git|latex" result/all-notes.txt
1:TODO: learn Shell
2:Git branch practice
3:Shell pipeline
4:TODO: review Git
5:LaTeX document practice
6:Git stash
7:TODO: write LaTeX report
8:Shell and Git review

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ sed -E 's/TODO:/DONE:/g' result/all-notes.txt > result/finished-notes.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ tr '[:upper:]' '[:lower:]' < result/finished-notes.txt \
| tr -cs '[:alpha:]' '\n' \
| sort \
| uniq -c \
| sort -nr \
• | head -n 10 > result/word-frequency.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ cat result/word-frequency.txt
  4 git
  3 shell
  3 done
  2 review
  2 practice
  2 latex
  1 write
  1 stash
  1 report
  1 pipeline

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ awk '{words += NF} END {print "lines =", NR, "words =", words}' \
result/finished-notes.txt
lines = 8 words = 24

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ find . -type f -name "*.txt" -printf "%p %s bytes\n" | sort
./daily/monday.txt  53 bytes
./daily/tuesday.txt 51 bytes
./daily/wednesday.txt 46 bytes
./result/all-notes.txt 150 bytes
./result/finished-notes.txt 150 bytes
./result/word-frequency.txt 145 bytes

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ chmod 640 daily/*.txt result/*.txt

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第一周/课后/shell-notes (main)
• $ du -ah . | sort -h
1.0K  ./daily/monday.txt
1.0K  ./daily/tuesday.txt
1.0K  ./daily/wednesday.txt
1.0K  ./result/all-notes.txt
1.0K  ./result/finished-notes.txt
1.0K  ./result/word-frequency.txt
3.0K  ./daily
3.0K  ./result
304K  ./1.png
310K  .
```

图 5 Shell 学习笔记处理与统计过程

15. `git restore --staged README.md`: 把文件移出暂存区但保留工作区修改。
16. `git commit --amend`: 补充遗漏内容或修改最近一次提交说明。
17. `git rebase -i HEAD~3`: 交互式整理最近三次本地提交。
18. `git blame README.md`: 查看每一行最后由哪次提交修改。
19. `git show HEAD:README.md`: 直接查看 HEAD 快照中的文件内容。
20. `git cat-file -p HEAD^{tree}`: 查看提交指向的树对象, 联系 Git 的对象模型。

练习与结果: 在独立仓库中完成分块暂存、三次提交、stash 恢复、restore 撤销、amend 补充提交和 v1.0 标签, 并用 log、show、blame 与 cat-file 验证了最终历史和文件内容, 相关操作记录见图 6 和图 7。

2.3 LaTeX 练习

1. 用 `ctexart` 写中文文档, 使中文能够正常排版。
2. 设置 `title`、`author`、`date`, 用 `maketitle` 生成标题。
3. 用 `section`、`subsection` 建立两级文档结构。
4. 用 `tableofcontents` 生成目录, 需要通过多轮编译更新页码。
5. 用 `itemize` 排版无序列表, 调整列表间距。
6. 用 `enumerate` 排版自动编号列表。
7. 用 `$E=mc^2$` 插入行内公式。
8. 用 `equation`、`label` 和 `eqref` 创建并引用带编号公式。
9. 用 `align` 对齐多行推导中的等号。
10. 用 `pmatrix` 排版矩阵, 并检查括号和列对齐。
11. 用 `cases` 排版分段函数。
12. 用 `tabular` 创建表格, 设置列对齐方式。
13. 引入 `booktabs`, 用 `toprule`、`midrule`、`bottomrule` 改进表格线条。
14. 用 `caption`、`label`、`ref` 完成表格编号与正文引用。
15. 引入 `graphicx`, 用 `includegraphics` 插入图片并按正文宽度缩放。
16. 引入 `hyperref`, 用 `href` 添加可点击的网页链接。
17. 用 BibTeX 建立 `.bib` 文件, 通过 `cite` 插入并生成参考文献。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git
• $ git init
Initialized empty Git repository in D:/desktop/第一周课后练习之git/.git/

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git branch -M main

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git config user.name "徐宜安"

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ printf "# Tool Notes\n\nInitial learning record.\n" > README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git status --short
?? README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git diff --staged
diff --git a/README.md b/README.md
new file mode 100644
index 0000000..50d5808
--- /dev/null
+++ b/README.md
@@ -0,0 +1,3 @@
+# Tool Notes
+
+Initial learning record.

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git commit -m "Add initial learning notes"
[main (root-commit) 20b3297] Add initial learning notes
1 file changed, 3 insertions(+)
create mode 100644 README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ printf "\n## Shell\nLearn pipes and redirection.\n" >> README.md; printf "\n## Git\nLearn staging and history.\n" >> README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git diff
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
diff --git a/README.md b/README.md
index 50d5808..8211201 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,9 @@
 # Tool Notes

Initial learning record.
+
+## Shell
+Learn pipes and redirection.
+
+## Git
+Learn staging and history.

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git add -p
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
diff --git a/README.md b/README.md
index 50d5808..8211201 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,9 @@
 # Tool Notes

Initial learning record.
+
+## Shell
+Learn pipes and redirection.
+
+## Git
+Learn staging and history.
(1/1) Stage this hunk [y,n,q,a,d,e,p,?]? y

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git diff --staged
diff --git a/README.md b/README.md
index 50d5808..8211201 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,9 @@
 # Tool Notes

Initial learning record.
+
+## Shell
+Learn pipes and redirection.
+
+## Git
+Learn staging and history.

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git commit -m "Add Shell and Git notes"
[main c5f9c51] Add Shell and Git notes
1 file changed, 6 insertions(+)
```

图 6 Git 练习：初始化、差异查看、分块暂存与提交

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ printf "\nTemporary unfinished paragraph.\n" >> README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git stash push -m "unfinished paragraph"
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state On main: unfinished paragraph

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git stash list
stash@{0}: On main: unfinished paragraph

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (b6181b0e53a584a96d7908e66aeb8cdc94e45a93)

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git restore README.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ printf "Missing Semester\nOI Wiki\n" > sources.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git add sources.md
warning: in the working copy of 'sources.md', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git commit -m "Add learning sources"
[main 150d278] Add learning sources
 1 file changed, 2 insertions(+)
 create mode 100644 sources.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ printf "Git documentation\n" >> sources.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git add sources.md
warning: in the working copy of 'sources.md', LF will be replaced by CRLF the next time Git touches it

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git commit --amend --no-edit
[main 013a569] Add learning sources
 Date: Mon Aug 31 00:10:43 2026 +0800
 1 file changed, 3 insertions(+)
 create mode 100644 sources.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git tag -a v1.0 -m "Complete first version"

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git status --short

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git log --graph --decorate --oneline
* 013a569 (HEAD -> main, tag: v1.0) Add learning sources
* c5f9c51 Add Shell and Git notes
* 20b3297 Add initial learning notes

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git show --stat v1.0
tag v1.0
Tagger: 徐宜安 <3989176408@qq.com>
Date:   Mon Aug 31 00:12:20 2026 +0800

Complete first version

commit 013a5693127f945a8b60d889e76d2ac6c15c4ca5 (HEAD -> main, tag: v1.0)
Author: 徐宜安 <3989176408@qq.com>
Date:   Mon Aug 31 00:10:43 2026 +0800

    Add learning sources

sources.md | 3 +++
1 file changed, 3 insertions(+)

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git blame README.md
^20b3297 (徐宜安 2026-08-31 00:05:14 +0800 1) # Tool Notes
^20b3297 (徐宜安 2026-08-31 00:05:14 +0800 2)
^20b3297 (徐宜安 2026-08-31 00:05:14 +0800 3) Initial learning record.
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 4)
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 5) ## Shell
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 6) Learn pipes and redirection.
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 7)
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 8) ## Git
c5f9c51e (徐宜安 2026-08-31 00:08:38 +0800 9) Learn staging and history.

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git show HEAD:sources.md
Missing Semester
OI Wiki
Git documentation

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
• $ git cat-file -p HEAD^{tree}
100644 blob 8211201ab590998b5a81a84f4c7e31a535282b0a README.md
100644 blob a035702aa6e002fc53b1dd05b8a335aaa258f42 sources.md

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/第一周课后练习之git (main)
```

图 7 Git 练习：stash、撤销、amend、标签与历史验证

练习与结果：使用 LaTeX 在一页中展示列表、公式、矩阵、分段函数、三线表、图片、超链接和 BibTeX 文献引用，最终 PDF 编号与交叉引用均正常，编译结果见图 8。完整代码见 课后/latex-planner/planner.tex，核心部分如下。

```
\documentclass[10pt,a4paper]{ctexart}
\usepackage{amsmath,booktabs,graphicx,caption,hyperref}
\begin{document}
\section{文档结构}
\begin{itemize}
  \item Shell 命令 \item Git 版本控制 \item LaTeX 排版
\end{itemize}
行内公式  $E=mc^2$ 。
\begin{equation} S=0.4A+0.6B \label{eq:score} \end{equation}
公式~\eqref{eq:score}表示加权成绩。
\begin{align} x+y&=10, & x-y&=2. \end{align}
\[
A=\begin{pmatrix}1&2\\3&4\end{pmatrix}, \quad
f(x)=\begin{cases}x,&x\geq0,\\-x,&x<0.\end{cases}
\]
\captionof{table}{学习安排}\label{tab:plan}
\begin{tabular}{ccc}
\toprule 工具&时间&状态\\ \midrule
Shell&2 h&完成\\ Git&3 h&完成\\ LaTeX&2 h&进行中\\
\bottomrule
\end{tabular}
\includegraphics[width=.3\textwidth]{example-image-a}
\href{https://missing-semester-cn.github.io/}{Missing Semester}
正文引用参考资料~\cite{missingsemester}.
\bibliographystyle{unsrt}
\bibliography{references}
\end{document}
```

代码 11 planner.tex 核心代码（排版参数略）

3 解题感悟

先记录下我这周学会了什么吧。之前大概只会一些基本的文件操作的命令，一直没搞明白 shell、bash、Terminal 这几个东西的关系，感觉都差不多，现在知道 Terminal 是输入文字用的交互地方，shell 解释命令沟通操作系统，bash 是其中一种 shell，还有关于脚本也有所了解了（之前好像还是挺怕这种东西的）。关于 git 我之前会在本地用些最简单的 add、commit 等命令，这次熟练了推送远程仓库（还知道怎么快速使用 Vscode 插件中的 gui 界面了，git 其它 gui 工具反而不喜欢不想用），以及关于分支之前没有好好学，这次收获了合并分支解决冲突。关于 Latex 之前有通识课结课论文用过，但是具体语法没那么熟，更习惯用语法更简单的 md 来写然后让 ai 给我转成 Latex 编译，这次更加熟悉了吧，还对不同配方更有了解。

我有在思考 ai 时代下我应该优先掌握什么（有的指令好难记啊我感觉不常用根本记不住）。首先肯定是得学过、跟着理解过这些工具，这些是基础，之后如果要用可以知道是什么有什么用，不常用的命令或者语法记不住可以问 ai 或者查文档教程。其次是一些程

LaTeX 基本功能展示

徐宜安 25060012033

2026 年 8 月 31 日

1 文档结构

1.1 列表

无序列表:

- Shell 命令
- Git 版本控制
- LaTeX 排版

有序列表:

1. 编写源文件
2. 编译并检查 PDF

1.2 数学公式

行内公式示例: $E = mc^2$ 。

带编号公式:

$$S = 0.4A + 0.6B \quad (1)$$

公式 (1) 表示加权成绩。

多行对齐:

$$x + y = 10, \quad (2)$$

$$x - y = 2. \quad (3)$$

矩阵:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

分段函数:

$$f(x) = \begin{cases} x, & x \geq 0, \\ -x, & x < 0. \end{cases}$$

2 图表与引用

表 1 展示学习安排。

表 1: 学习安排

工具	时间	状态
Shell	2 h	完成
Git	3 h	完成
LaTeX	2 h	进行中

图 1 展示图片插入效果。



图 1: 图片示例

3 链接与文献

课程参考资料:

[Missing Semester 中文版](#)

正文引用参考资料 [1]。

参考文献

[1] MIT. The missing semester of your cs education. <https://missing-semester-cn.github.io/>, 2026.

图 8 LaTeX 基本功能展示的编译结果

序员必备的技能了, 比如基本的 shell 命令、git 命令, 这些肯定要牢记。现在我可能 awk、sed 这种复杂命令还不太熟, Latex 让我直接写估计也只能写个结构, 但是感觉更值得学的还是 git, 把分支学明白。

最近越来越体会到学计算机需要接触自己陌生的新事物, 学技术, 也希望自己能从这四周的课开始好好实践吧。

第一周课上实验、课后练习及实验报告均已提交至 Git 仓库, 相关提交记录见图 9。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础 (main)
● $ git log --oneline --graph
* c0fd71c (HEAD -> main, origin/main, origin/HEAD) 完成课后练习并放进实验报告中
* c134e78 设计出实验报告模板以及写完第一周实验报告大部分, 待插入课后练习图片和commit记录
* 9e145d9 略微优化了q01和q04, 补充截图
* 8c46a5b 第一周课上q04任务完成
* f594133 第一周课上q03任务完成
* d49ac02 第一周课上q02任务完成
* 47bead7 第一周课上q01任务完成
```

图 9 第一周实验及报告的 Git 提交记录